

Final project communication networks

Mor Romano - 207707191

Nickolai Korgan - 342854965

Liza Behak - 206531972

System architecture.....	3
DNS Module – Project Explanation.....	3
Overview.....	3
Zone Database.....	4
DNS Packet Parsing.....	5
DNS Header Parsing.....	5
DNS Question Parsing.....	6
DNS Response Construction.....	7
Answer Section.....	7
SOA Records.....	8
UDP Transport Layer.....	9
DNS Server Logic.....	9
DNS Client (DNSService).....	12
Query construction.....	12
System Workflow.....	12
Key Concepts Demonstrated.....	13
JSON-based DNS Implementation.....	13
Overview.....	14
Zone Database.....	14
JSON DNS Protocol.....	15
Request Processing.....	16
1. DNS Lookup Request.....	16
2. Add DNS Record.....	17
JSON DNS Server.....	18
Request Handling.....	18
UDP Transport Layer.....	19
DNS Client.....	20
Server Execution.....	21
System Workflow.....	21
Advantages of JSON Version.....	22
Summary of Prompts Used During Development - DNS:.....	23
1. DNS Protocol and Packet Structure.....	23
2. UDP Networking in Python.....	24
3. Binary Data Handling in Python.....	24
4. JSON Database Management.....	25
5. DNS Request Handling Logic.....	26

6. Testing Prompts.....	26
FTP Module – Project Explanation.....	27
Overview.....	27
Main Components.....	27
Overall Architecture.....	28
Control Channel Design.....	28
Session Handling and Concurrency.....	28
File Listing and Filtering.....	29
JSON-Based Control Protocol.....	29
Transfer Negotiation.....	29
TCP Transfer Path.....	30
RUDP Transfer Path.....	30
Custom RUDP Architecture.....	31
RUDP Transfer Lifecycle.....	31
Client-Side FTP Logic.....	32
Why This Architecture Was Used.....	32
Short Summary.....	33
Summary of Prompts Used During Development - FTP and RUDP:.....	33
1. Multi-threaded Server Architecture.....	33
2. Control vs. Data Channel Logic.....	34
3. File System and Filtering.....	34
4. Binary File Handling and Chunking.....	34
5. RUDP (Reliable UDP) Implementation Logic.....	34
6. Protocol Integration and Testing.....	34
DHCP.....	36
Message Format & Fields.....	36
Modules: (Binary Version).....	38
Modules: (JSON Version).....	39
The D.O.R.A Process:.....	39
Implementation Challenges & Simulations.....	41
Code Explained.....	41
Summary of Prompts Used During Development - DHCP:.....	51
Workflow.....	51
DHCP.....	51
DNS.....	53
FTP.....	54
Dissectors.....	55
DHCP_JSON.....	55
DNS_JSON.....	56
FTP (FOJP).....	57
CRUDP.....	58
Wireshark Flows.....	60
RFC.....	60
JSON.....	62

Q & A.....	64
TCP vs QUIC.....	64
Vegas vs CUBIC.....	65
What is the BGP Protocol?.....	66
How is BGP Different From OSPF?.....	66
Differences Between ARP and DNS.....	67

System architecture

The project is built as a small end-to-end application-layer network stack composed of DHCP, DNS, FTP, and a custom RUDP transport. The goal is to simulate how a client joins a network, discovers infrastructure services, resolves a service name, and finally downloads a file.

The architecture supports two parallel implementation styles:

- an RFC-inspired path, where DHCP and DNS are encoded in binary structures similar to their real protocol formats
- a JSON-based path, where the same logical flow is implemented with human-readable JSON messages for easier debugging and Wireshark inspection

In both paths, the final application is the same FTP-like file transfer service. The client first receives an IP address and DNS server address from DHCP, then resolves the FTP server hostname through DNS, and finally connects to the FTP server to browse and download media files. The download itself can be carried out either over TCP or over the custom RUDP protocol.

DNS Module – Project Explanation

Overview

The DNS component of the project implements a simplified authoritative DNS server and a DNS client resolver using the UDP protocol.

The goal of this module is to demonstrate how DNS works internally by manually parsing DNS packets, querying a zone database, and constructing a valid DNS response in binary format.

The system includes the following major components:

1. ZoneDatabase – manages DNS records stored in a JSON file.
2. DNS protocol parser and response builder – parses incoming DNS queries and builds binary DNS responses.

3. UDP transport layer – handles network communication between the DNS client and server.
4. DNSServer – the core server logic that processes DNS requests.
5. DNSService (client) – sends DNS queries and extracts the resolved IP address from the response.

The server listens on UDP port 8053 and behaves like a simplified authoritative DNS server for the zones defined in **dns.json**.

Zone Database

The DNS records are stored in a JSON file (**dns.json**) that acts as the server's **zone database**.

Example entries:

```
{  
  "example.com": "127.0.0.1",  
  "www.example.com": "127.0.0.2",  
  "mail.example.com": "127.0.0.3",  
  "google.com": "142.250.75.110"  
}
```

Each entry maps a **domain name** to an **IPv4 address**.

The ZoneDatabase class is responsible for:

- Loading DNS records from the JSON file.
- Extracting the **zone name** from domains.
- Looking up IP addresses.
- Providing SOA (Start of Authority) information when required.

Zone Extraction

To determine whether the server is authoritative for a request, the system extracts the zone from the domain name.

Example:

`www.google.com → google.com`

`mail.example.com → example.com`

This allows the server to determine if it should answer the query or refuse it.

DNS Packet Parsing

DNS messages follow a strict binary format.

Each DNS packet contains:

HEADER | QUESTION | ANSWER | AUTHORITY | ADDITIONAL

The project manually parses this structure instead of using external libraries.

DNS Header Parsing

The **DNSHeader** class parses the **first 12 bytes** of the DNS message.

These fields include:

Field	Size	Description
Transaction ID	2 bytes	Used to match response with request
Flags	2 bytes	Defines query/response behavior
QDCOUNT	2 bytes	Number of questions
ANCOUNT	2 bytes	Number of answers
NSCOUNT	2 bytes	Authority records
ARCOUNT	2 bytes	Additional records

Example from the code:

```
self.transaction_id = int.from_bytes(self.raw_data[0:2], "big")
```

```
self.flags = int.from_bytes(self.raw_data[2:4], "big")
```

The flags field is then decomposed into individual bits:

- QR – query or response
- OPCODE – operation type
- AA – authoritative answer
- TC – truncation
- RD – recursion desired
- RA – recursion available
- RCODE – response code

DNS Question Parsing

The **DNSQuestion** class extracts the domain name being queried.

DNS encodes domain names using **length-prefixed labels**.

Example: www.example.com

Binary format: 3 'www' 7 'example' 3 'com' 0

Example implementation:

```
length = self.raw_data[i]
```

```
labels.append(self.raw_data[i+1:i+1+length].decode())
```

The parser reconstructs the domain name and extracts:

- **QNAME** – domain name
- **QTYPE** – record type (A, AAAA, etc.)
- **QCLASS** – class (usually IN)

DNS Response Construction

The **DNSResponseBuilder** class constructs a valid DNS response.

The response includes:

HEADER | QUESTION | ANSWER | AUTHORITY

Header Construction

The server builds the header using **struct.pack**:

```
struct.pack("!HHHHHH", ...)
```

Key modifications:

- QR bit set to **1** (response)
- AA bit set if authoritative
- RCODE depending on result

Answer Section

If the requested record exists, the server returns an **A record** containing the IPv4 address.

Structure:

NAME | TYPE | CLASS | TTL | RDLENGTH | RDATA

Example code:

```
answer = struct.pack(  
    "!HHHLH4s",  
    aname,  
    atype,
```

```
aclass,  
attl,  
ardlength,  
ardata  
)
```

Where:

RDATA = IPv4 address converted using `inet_aton()`

SOA Records

If the domain does not exist in the zone, the server returns an **NXDOMAIN response**.

In this case the authority section includes a **Start of Authority (SOA)** record.

The SOA record contains:

- Primary nameserver
- Responsible email
- Serial number
- Refresh time
- Retry time
- Expiration
- Minimum TTL

Example values:

mname: ns1.example.com

rname: hostmaster.example.com

serial: 13121312

UDP Transport Layer

The **UdpTransport** class handles network communication.

The server uses **UDP sockets**, which is the standard DNS transport protocol.

Key operations:

Initialize socket

```
socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Receive query

```
data, addr = sock.recvfrom(buffer_size)
```

Send response

```
sock.sendto(data, addr)
```

The implementation also supports:

- packet loss simulation
- artificial network delay

These features help test robustness.

DNS Server Logic

The **DNSServer** class implements the main request handling logic.

Steps performed when a request arrives:

1. Validate DNS packet

2. Parse query using **DNSResponseBuilder**
3. Extract domain name and record type
4. Determine if the server is authoritative
5. Lookup domain in **ZoneDatabase**
6. Construct appropriate DNS response

Possible outcomes:

Successful lookup

RCODE = 0

Answer section contains IP

```
(.venv) (base) just_another_one@Mac DNS_binary % dig @127.0.0.1 -p 8053 example.com A

; <<>> DiG 9.10.6 <<>> @127.0.0.1 -p 8053 example.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22905
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 300     IN      A      127.0.0.1

;; Query time: 0 msec
;; SERVER: 127.0.0.1#8053(127.0.0.1)
;; WHEN: Thu Mar 12 13:20:53 IST 2026
;; MSG SIZE rcvd: 45
```

Domain does not exist

RCODE = 3 (NXDOMAIN)

SOA record returned

```
(.venv) (base) just_another_one@Mac DNS_binary % dig @127.0.0.1 -p 8053 unknown.example.com A

; <<>> DiG 9.10.6 <<>> @127.0.0.1 -p 8053 unknown.example.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 13762
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;unknown.example.com.          IN      A

;; AUTHORITY SECTION:
example.com.                   300     IN      SOA     ns1.example.com. hostmaster.example.com. 13121312 3600 600 86400 300

;; Query time: 0 msec
;; SERVER: 127.0.0.1#8053(127.0.0.1)
;; WHEN: Thu Mar 12 13:24:57 IST 2026
;; MSG SIZE rcvd: 121
```

Unsupported record type

RCODE = 0

No answer section

```
(.venv) (base) just_another_one@Mac DNS_binary % dig @127.0.0.1 -p 8053 example.com AAAA

; <<>> DiG 9.10.6 <<>> @127.0.0.1 -p 8053 example.com AAAA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 64953
;; flags: qr aa rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;example.com.                  IN      AAAA

;; AUTHORITY SECTION:
example.com.                   300     IN      SOA     ns1.example.com. hostmaster.example.com. 13121312 3600 600 86400 300

;; Query time: 0 msec
;; SERVER: 127.0.0.1#8053(127.0.0.1)
;; WHEN: Thu Mar 12 13:26:06 IST 2026
;; MSG SIZE rcvd: 113
```

Query for unknown zone

RCODE = 5 (REFUSED)

```
(.venv) (base) just_another_one@Mac DNS_binary % dig @127.0.0.1 -p 8053 openai.com A

; <<>> DiG 9.10.6 <<>> @127.0.0.1 -p 8053 openai.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 40670
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;openai.com.                   IN      A

;; Query time: 0 msec
;; SERVER: 127.0.0.1#8053(127.0.0.1)
;; WHEN: Thu Mar 12 13:28:22 IST 2026
;; MSG SIZE rcvd: 28
```

DNS Client (DNSService)

The `DNSService` class implements a simple DNS resolver.

It manually builds DNS queries and sends them to the DNS server.

Query construction

```
header = struct.pack("!HHHHHH", tx_id, 0x0100, 1, 0, 0, 0)
```

The query includes:

- Transaction ID
- Standard query flags
- One question

Sending request

```
sock.sendto(query, (dns_server_ip, 8053))
```

Receiving response

The resolver extracts the IPv4 address from the response:

```
socket.inet_ntoa(data[-4:])
```

System Workflow

The complete process works as follows:

1. Client sends DNS query.
2. Server receives packet via UDP.
3. Server parses DNS header and question.

4. Server checks zone authority.
5. Server searches domain in **ZoneDatabase**.
6. Server constructs binary DNS response.
7. Response is sent back to client.
8. Client extracts IP address.

Key Concepts Demonstrated

This project demonstrates several important networking concepts:

- DNS packet structure
- Binary protocol parsing
- UDP socket communication
- Authoritative DNS behavior
- Domain name encoding
- Resource record construction
- Error handling in network protocols

JSON-based DNS Implementation

Overview

In addition to the binary DNS implementation, the project includes a **simplified DNS system using JSON messages**.

This version was designed to demonstrate DNS resolution logic **without the complexity of the DNS binary protocol**.

Instead of parsing raw DNS packets, the system exchanges **JSON messages over UDP**.

This makes the protocol easier to test and debug while still demonstrating:

- UDP networking
- DNS-style name resolution
- Database-backed domain lookup
- Request/response protocol design

The JSON-based DNS consists of three main components:

1. **DNS client (client.py)** – sends DNS queries
2. **JSON DNS server** – processes requests and returns results
3. **Zone database (dns.json)** – stores domain name to IP mappings

Communication occurs over **UDP port 9000**.

Zone Database

Domain records are stored in a JSON file called **dns.json**.

Example records:

```
{
```

```
"example.com": "127.0.0.1",  
"www.example.com": "127.0.0.2",  
"mail.example.com": "127.0.0.3",  
"google.com": "142.250.75.110",  
"test.com": "5.6.7.8"  
}
```

Each entry maps:

domain name → IPv4 address

The database is managed by the **ZoneDatabase** class.

Database Responsibilities

The class provides several key functions:

Method	Purpose
<code>_load()</code>	Loads records from <code>dns.json</code>
<code>_save()</code>	Writes updated records to the file
<code>lookup()</code>	Returns the IP for a domain
<code>is_in_zone()</code>	Checks if a domain exists
<code>add_record()</code>	Adds a new domain record

Example lookup:

```
ip = database.lookup("google.com")
```

JSON DNS Protocol

Unlike real DNS packets, this version uses **JSON messages**.

Example query:

```
{  
  
  "url": "google.com"  
  
}
```

Example response:

```
{  
  
  "ip": "142.250.75.110"  
  
}
```

If the domain does not exist:

```
{  
  
  "ip": null  
  
}
```

Errors are returned as:

```
{  
  
  "error": "Invalid request format"  
  
}
```

Request Processing

The core logic is implemented in the function:

```
resolve_request(database, request_dict)
```

This function handles two types of requests.

1. DNS Lookup Request

The simplest request resolves a domain name to an IP address.

Example request:

```
{
```



```
"url": "google.com"
}
```

Processing steps:

1. Extract the "**url**" field
2. Normalize the domain name
3. Check if it exists in the zone database
4. Return the corresponding IP address

Example result:

```
{
  "ip": "142.250.75.110"
}
```

If the domain does not exist:

```
{
  "ip": null
}
```

2. Add DNS Record

The system also supports **adding new DNS records dynamically**.

Example request:

```
{
  "url": "newdomain.com",
  "ip": "1.2.3.4"
}
```

```
}
```

Processing steps:

1. Validate the IP address using **`socket.inet_aton()`**
2. Normalize the domain name
3. Add the record to the database
4. Save the updated database to **`dns.json`**

Example result:

```
{  
  "ip": "1.2.3.4"  
}
```

JSON DNS Server

The JSON DNS server is implemented in the **`json_dns_server`** class.

This component receives raw UDP data, decodes JSON requests, and returns JSON responses.

Request Handling

Steps performed by the server:

1. Receive raw bytes from UDP socket
2. Decode bytes into UTF-8 string
3. Parse JSON request

4. Call **resolve_request()**

5. Convert result back to JSON

6. Send response to client

Example implementation:

```
decoded_data = raw_data.decode("utf8")
```

```
data = json.loads(decoded_data)
```

```
resolved_data = resolve_request(self.zone_database, data)
```

The response is encoded before sending:

```
json.dumps(resolved_data).encode("utf8")
```

UDP Transport Layer

Network communication is handled by the **UdpTransport** class.

This class manages:

- socket creation
- packet sending
- packet receiving
- optional packet loss simulation
- artificial network delay

Socket Initialization

```
socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Receiving packets

```
data, addr = self.sock.recvfrom(self.buffer_size)
```

Sending responses

```
self.sock.sendto(data, addr)
```

The server listens on:

127.0.0.1:9000

DNS Client

The client is implemented in **client.py** using the **DNSService** class.

This component constructs DNS queries and sends them to the server.

Query Creation

The client builds a DNS query manually using **struct.pack**.

Example header:

```
header = struct.pack("!HHHHHH", tx_id, 0x0100, 1, 0, 0, 0)
```

The domain name is encoded as a sequence of labels.

Example:

google.com

Encoded as:

6 google 3 com 0

Sending Query

```
sock.sendto(query, (self.dns_server_ip, 8053))
```

Receiving Response

The client extracts the IP address from the last four bytes:

```
socket.inet_ntoa(data[-4:])
```

Server Execution

The DNS server starts with:

```
run_dns_server("127.0.0.1", 9000)
```

Execution flow:

1. Initialize UDP socket
 2. Load DNS database
 3. Wait for incoming packets
 4. Process request
 5. Send response
-

System Workflow

The JSON DNS system operates as follows:

1. Client sends JSON request via UDP
2. Server receives the packet
3. Server parses JSON data

4. Server queries the zone database
 5. Server generates JSON response
 6. Client receives response and extracts IP
-

Advantages of JSON Version

The JSON-based DNS implementation provides several advantages for development and testing:

- easier debugging compared to binary DNS packets
- simpler protocol structure
- flexible request format
- dynamic DNS record updates
- human-readable communication

This version served as a **prototype implementation** before building the full **binary DNS protocol version**.

Summary of Prompts Used During Development - DNS:

During the development of the DNS project, AI assistance was used mainly to support the implementation of networking logic, DNS packet handling, database operations, and testing. The prompts were used to solve specific programming problems and guide the development of different components of the system.

The prompts can be grouped into several categories.

1. DNS Protocol and Packet Structure

Several prompts were used to understand the structure of DNS packets and how queries and responses are encoded.

Example prompts included:

- *"What is the structure of a DNS packet?"*
- *"How is a DNS header encoded in binary?"*
- *"How are domain names encoded in DNS messages?"*
- *"What does each DNS header flag mean (QR, AA, RD, RCODE)?"*
- *"What is the format of the DNS answer section?"*
- *"How does the DNS compression pointer work (0xC00C)?"*
- *"What is the difference between NXDOMAIN and NOERROR?"*
- *"When should a DNS server return REFUSED?"*

These prompts helped implement the binary DNS protocol and construct valid DNS responses.

2. UDP Networking in Python

AI assistance was used to implement the UDP server responsible for receiving and sending DNS messages.

Example prompts included:

- *“How do I create a UDP server in Python?”*
- *“How can I send and receive UDP packets using Python sockets?”*
- *“How do I bind a socket to a specific IP and port?”*
- *“How do I set a timeout for a UDP socket?”*
- *“How do I simulate packet loss in a UDP server?”*
- *“How can I add artificial network delay in Python?”*
- *“How do I handle socket timeout exceptions?”*

These prompts helped build the **UdpTransport** component used by the DNS server.

3. Binary Data Handling in Python

Since the DNS protocol is binary, prompts were used to work with byte-level data.

Example prompts included:

- *“How do I convert bytes to integers in Python?”*

- *“How does **struct.pack** work in Python?”*
- *“How do I unpack DNS header fields from raw bytes?”*
- *“How do I encode an IPv4 address into DNS RDATA format?”*
- *“How can I convert an IP address string into binary using **socket.inet_aton()**?”*
- *“How do I read specific bits from a binary flag field?”*
- *“How do I extract specific bytes from a network packet?”*

These prompts helped implement the parsing and construction of DNS packets.

4. JSON Database Management

The project stores DNS records in a JSON-based zone database. Prompts were used to implement loading, saving, and updating records.

Example prompts included:

- *“How do I load JSON data from a file in Python?”*
- *“How do I write JSON data back to a file?”*
- *“How can I update a JSON dictionary and save it?”*
- *“How can I normalize domain names before storing them?”*
- *“How do I validate IPv4 addresses in Python?”*

- *“How do I handle missing JSON files?”*

These prompts supported the implementation of the **ZoneDatabase** class.

5. DNS Request Handling Logic

Prompts were used to help design the logic for handling incoming DNS queries.

Example prompts included:

- *“How should a DNS server process a query step by step?”*
- *“How do I check if a domain exists in my zone database?”*
- *“How should the server behave if a domain is not found?”*
- *“How should the server behave when the requested record type is unsupported?”*
- *“How should I build a DNS response when no answer exists?”*
- *“How do I return an SOA record in the authority section?”*

These prompts helped define the request-handling workflow of the DNS server.

6. Testing Prompts

Several prompts were used to test the DNS server and verify that it behaves correctly under different conditions.

Example prompts included:

- *“How can I test my DNS server using the **dig** command?”*

- *“What commands can I use to query a DNS server running on a custom port?”*
- *“How do I test NXDOMAIN responses?”*
- *“How do I test unsupported record types using **dig**?”*
- *“How do I send UDP JSON requests from Python?”*
- *“How can I simulate malformed requests?”*
- *“How do I check if my DNS response header is correct?”*

These prompts helped validate that the server correctly handles multiple DNS scenarios.

FTP Module – Project Explanation

Overview

The FTP component of the project implements a simplified, FTP-inspired file transfer system that allows a client to browse and download media files from a server. The module uses a two-channel architecture similar to classic FTP: a persistent TCP control channel is used for commands and negotiation, while a separate temporary data channel is created for the actual file transfer.

A main feature of this implementation is that the data channel can work in two different modes:

- TCP, which relies on the operating system's built-in reliability
- RUDP, a custom Reliable UDP protocol implemented as part of the project

This makes the FTP module a hybrid design: the control plane is always TCP, but the payload transfer can be done either with standard TCP or with a custom reliability layer built on top of UDP.

Main Components

The FTP module is built from the following files:

- `ftp_server.py` – the server-side FTP logic
- `ftp_service.py` – the client-side FTP logic

- rudp_server.py – sends files using the custom RUDP protocol
- rudp_service.py – receives files using the custom RUDP protocol
- rudp_protocol.py – defines the RUDP packet format and flags

Overall Architecture

The FTP module is part of the larger project flow:

1. The client receives an IP address from DHCP
2. The client resolves the FTP server hostname using DNS
3. The client opens a TCP control connection to the FTP server on port 2121
4. The client requests the file menu
5. The client chooses a file and selects TCP or RUDP
6. The server creates a temporary data port and sends the port number back to the client
7. The client connects to that temporary port and downloads the file

This architecture separates control traffic from data traffic. The control connection stays open for the session, while the data connection exists only for the duration of a single file transfer.

Control Channel Design

The server listens on TCP port 2121 instead of port 21 so it does not interfere with a real FTP service on the machine.

Example server setup:

```
welcome_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
welcome_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
welcome_sock.bind((self.host, self.control_port))
welcome_sock.listen(5)
```

Unlike full RFC 959 FTP, this project does not implement commands such as USER, PASS, LIST, or RETR. Instead, it uses a simplified protocol:

- the client sends "ftp" to request the menu
- the client sends "quit" to end the session
- file requests and server responses are exchanged as JSON messages

This simplified design makes the protocol easier to debug and easier to inspect in Wireshark.

Session Handling and Concurrency

The FTP server is multi-threaded. Every time a client connects to the control port, the server starts a new thread to handle that session.

Example:

```
while True:
    control_conn, addr = welcome_sock.accept()
    threading.Thread(target=self.handle_client_session, args=(control_conn,
addr)).start()
```

This allows multiple clients to connect, browse files, and start transfers without blocking the main listening socket.

File Listing and Filtering

The server only exposes files from the `server_files` directory. It filters the results so that only specific media formats are available:

- .mp3
- .jpg
- .JPG
- .jpeg
- .gif
- .mp4

Example:

```
valid_extensions = ('.mp3', '.jpg', '.JPG', '.jpeg', '.gif', '.mp4')
return [f for f in os.listdir(self.file_dir) if f.endswith(valid_extensions)][0:10]
```

The list is limited to the first 10 valid files, which keeps the menu small and focused on demonstrating file transfer rather than implementing a complete file browser.

JSON-Based Control Protocol

When the client sends the command "ftp", the server responds with a JSON menu of available files.

Example server code:

```
if data.lower() == 'ftp':
    file_list = self.get_file_list()
    menu_data = json.dumps({"type": "MENU", "files": file_list})
    control_conn.send((menu_data + "\n").encode("utf-8"))
```

Example menu message:

```
{"type": "MENU", "files": ["Skulls.jpg", "Jimmy.mp3"]}
```

After selecting a file and a transfer mode, the client sends a JSON request to the server.

Example client code:

```
control_sock.send(json.dumps({
    "filename": selected_file,
    "mode": mode
}).encode('utf-8'))
```

Example request:

```
{"filename": "Jimmy.mp3", "mode": "RUDP"}
```

If the file exists, the server replies with a ready message that includes the dynamically assigned data port. If the file does not exist, the server returns an error message.

Transfer Negotiation

A key design decision in this module is that the server does not transfer the file over the control socket. Instead, it opens a new temporary socket for the data channel and informs the client which port to use.

Example:

```
response = json.dumps({"status": "ready", "mode": "TCP", "data_port": data_port})
control_conn.send(response.encode('utf-8'))
```

Example ready message:

```
{"status": "ready", "mode": "TCP", "data_port": 50341}
```

This is similar to classic FTP, where commands and file data are transferred on separate channels.

TCP Transfer Path

If the client selects TCP mode, the server creates a new TCP socket and binds it to port 0, which tells the operating system to choose an available ephemeral port automatically.

Example:

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as data_welcome_sock:
    data_welcome_sock.bind((self.host, 0))
    data_port = data_welcome_sock.getsockname()[1]
    data_welcome_sock.listen(1)
```

After the client connects to that port, the server reads the file in binary mode and sends it in 8192-byte chunks:

```
with data_conn, open(file_path, "rb") as f:
    while chunk := f.read(8192):
        data_conn.sendall(chunk)
```

On the client side, the file is received using repeated `recv()` calls and written to a local file:

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ds:
    ds.connect((self.server_ip, port))
    with open(path, "wb") as f:
        while chunk := ds.recv(8192):
            f.write(chunk)
```

In this mode, all reliability is handled by TCP itself, including ordering, retransmission, and flow control.

RUDP Transfer Path

If the client selects RUDP mode, the control connection still runs over TCP, but the file data is transferred using the project's custom Reliable UDP protocol.

The FTP server creates an `RUDPServer` object, gets its UDP port, and sends that port to the client:

```
rudp_handler = RUDPServer(self.host, 0)
data_port = rudp_handler.sock.getsockname()[1]
response = json.dumps({"status": "ready", "mode": "RUDP", "data_port": data_port})
control_conn.send(response.encode('utf-8'))
```

The server then reads the file into memory and passes the bytes to the RUDP layer:

```
with open(file_path, "rb") as f:
    file_bytes = f.read()
```

```
rudp_handler.send_file(file_bytes, client_addr)
```

Custom RUDP Architecture

The RUDP protocol adds reliability to UDP by defining its own packet structure. Each packet contains:

- sequence number
- total number of packets
- flags field
- payload data

Header definition:

```
HEADER_FORMAT = "!IIB3x"
HEADER_SIZE = struct.calcsize(HEADER_FORMAT)
MAX_PAYLOAD_SIZE = 1450
```

This creates a 12-byte header:

- 4 bytes – sequence number
- 4 bytes – total packets
- 1 byte – flags
- 3 bytes – padding

The protocol uses three important control flags:

- SYN – used to start the transfer
- ACK – used to acknowledge received packets
- FIN – used to signal the end of the transfer

Example:

```
FIN_FLAG = 0x01
ACK_FLAG = 0x02
SYN_FLAG = 0x04
```

RUDP Transfer Lifecycle

The RUDP client begins the session by sending a SYN packet to the server:

```
trigger = RUDPProtocol.create_packet(0, 0, b"", flags=RUDPProtocol.SYN_FLAG)
self.sock.sendto(trigger, self.server_addr)
```

The server waits for this trigger before sending file data:

```
data, actual_client_addr = self.sock.recvfrom(1024)
_, _, flags, _ = RUDPProtocol.parse_packet(data)
if not (flags & RUDPProtocol.SYN_FLAG):
    return
```

The server splits the file into packets based on the maximum payload size:

```
packets = [file_bytes[i:i + RUDPProtocol.MAX_PAYLOAD_SIZE]
            for i in range(0, len(file_bytes), RUDPProtocol.MAX_PAYLOAD_SIZE)]
```

It then sends packets using a sliding window algorithm:

```
while next_to_send <= upper_bound:
    packet = RUDPProtocol.create_packet(next_to_send, self.total_packets,
    packets[next_to_send])
    self.sock.sendto(packet, actual_client_addr)
    next_to_send += 1
```

The client stores each received packet by sequence number and sends an ACK:

```
if seq not in received_chunks:
    received_chunks[seq] = chunk

ack_packet = RUDPProtocol.create_ack(seq)
self.sock.sendto(ack_packet, addr)
```

The server tracks the highest ACK received and uses that information to decide when retransmission may be needed:

```
if flags & RUDPProtocol.ACK_FLAG:
    with self.lock:
        self.last_ack = max(self.last_ack, seq)
```

At the end of the transfer, the server sends a FIN packet several times to improve the chance that the client receives the termination signal:

```
fin_packet = RUDPProtocol.create_packet(self.total_packets, self.total_packets, b"",
flags=RUDPProtocol.FIN_FLAG)
for _ in range(5):
    self.sock.sendto(fin_packet, actual_client_addr)
```

Client-Side FTP Logic

The FTP client is interactive. It:

8. connects to the control socket
9. requests the menu
10. displays the available files
11. asks the user to choose a file number
12. asks the user to choose TCP or RUDP
13. receives the server's ready response
14. downloads the file using the selected transport

The downloaded file is saved locally using the pattern:

```
out_path = f"downloaded_{selected_file}"
```

This makes it easy to verify that the transfer completed successfully without overwriting the original file on the server.

Why This Architecture Was Used

This architecture was chosen because it demonstrates several important networking ideas in one application:

- persistent TCP control sessions
- separation of control and data channels
- dynamic port negotiation
- JSON-based application-layer messaging
- binary file transfer
- comparison between built-in reliability and custom reliability

Using TCP for the control plane guarantees that commands and negotiation messages arrive in order.

Using TCP or RUDP for the data plane allows the project to compare two different reliability models: one provided by the operating system and one implemented manually in the application.

Short Summary

The FTP module uses a split architecture with a TCP control channel and a temporary data channel for file transfer. The client first requests a file menu, then selects a file and a transfer mode. The server responds with a dynamic port number and the client downloads the file using either TCP or the custom RUDP protocol. TCP mode relies on built-in transport reliability, while RUDP mode implements reliability manually using sequence numbers, acknowledgements, retransmission behavior, and a final FIN packet. This design allows the project to demonstrate both classic FTP-style channel separation and custom transport-layer logic in the same system.

Summary of Prompts Used During Development - FTP and RUDP:

During the development of the FTP and RUDP modules, AI assistance was primarily used to implement multi-threaded session management, dynamic port negotiation, and the logic required to bridge TCP control channels with custom RUDP data transfers. These prompts guided the creation of a system that can handle both standard stream-based and custom reliability protocols.

The prompts are grouped into the following categories:

1. Multi-threaded Server Architecture

Prompts were used to ensure the server could handle multiple clients simultaneously without blocking the main control port.

- "How do I use `threading.Thread` to handle multiple client connections in a Python TCP server?"
- "How can I ensure the main server loop continues to `accept()` new connections while a file transfer is in progress?"
- "What is the best way to pass a connection socket to a thread and ensure it is closed properly using a `with` statement?"

2. Control vs. Data Channel Logic

Assistance was used to design the "out-of-band" signaling where the control channel negotiates a port for the data channel.

- "How do I bind a socket to port 0 to let the operating system assign an available ephemeral port?"
- "How do I retrieve the assigned port number from a socket using `getsockname()`?"
- "How should a client transition from a TCP control connection to a separate RUDP data connection?"

3. File System and Filtering

Prompts helped implement secure and restricted file browsing within the server's directory.

- "How do I list only files with specific extensions like .mp3 or .jpg in Python?"
- "How do I limit the results of `os.listdir()` to the first 10 items?"
- "How do I safely join file paths using `os.path.join` to prevent directory traversal attacks?"

4. Binary File Handling and Chunking

Since the FTP module handles media files, prompts were used to manage binary data and memory-efficient transfers.

- "How do I read a large binary file in chunks of 8192 bytes to avoid high memory usage?"
- "What is the difference between `send()` and `sendall()` when transferring files over TCP?"
- "How can I convert an entire file into a byte array to pass it to a custom UDP-based protocol?"

5. RUDP (Reliable UDP) Implementation Logic

Several prompts focused on mimicking TCP-like reliability features over a connectionless UDP socket.

- "How do I implement a 'Stop-and-Wait' ARQ protocol using UDP sockets?"
- "How can I use `struct.pack` to add sequence numbers and checksums to a UDP payload?"
- "How do I set a timeout on a UDP socket to trigger a retransmission of a lost packet?"

6. Protocol Integration and Testing

Prompts were used to verify that the JSON-based control messages and the binary data transfers worked in tandem.

- "How do I encode a Python dictionary into a JSON string and add a newline character for delimited streaming?"
- "How do I simulate packet loss or network delay to test the robustness of my RUDP implementation?"
- "How can I use `socket.inet_ntoa()` to extract an IP address from binary data received by the client?"

DHCP

DHCP (Dynamic Host Configuration Protocol) is an App Layer Protocol used for allocation of IP addresses to a fellow computer who just connected a new local network.

The address is used for communicating with fellow computers and the gateway.

In a real DHCP environment what's often done is setting up a socket at the link layer, and rewriting the entire headers of the Ethernet, IP and UDP and sending the payload as the DHCP header.

This is done because when a computer boots up he doesn't have an IP address, so he can't communicate over UDP, and communicate over IP, only through the link layer, using its own MAC address.

Since this process can't be done in that level at our project we didn't need to write the ethernet, ip and udp headers from scratch, only the DHCP headers are written and sent.

Message Format & Fields

DHCP is based on the BOOTP protocol format, which is shown here:

op (1)	htype (1)	hlen (1)	hops (1)
xid (4)			
secs (2)		flags (2)	
ciaddr (4)			
yiaddr (4)			
siaddr (4)			
giaddr (4)			
chaddr (16)			
sname (64)			
file (128)			
options (312)			

1. op - operation code:

Determine the kind of the message.

- DISCOVER - 0x01
- OFFER - 0x02
- REQUEST - 0x03
- ACK - 0x05

2. htype - hardware type: commonly set as 0x01 for Ethernet

3. hlen - hardware length - commonly set as 6, for MAC address

4. Hops - number of hops a message can go through DHCP Agents without being thrown, acts the DHCP TTL.

5. xid - Transaction ID, used to determine the current session

6. secs - the lease time of the allocated IP, counted in seconds

7. flags - a legacy BOOTP fields, used for determining if the message is sent over a unicast or a broadcast, only the MSB is used, rest is reserved (set to zero)

8. ciaddr - client IP address - the client's IP address, is set to zero for most of the DORA process

9. yiaddr - "your" IP address, the IP that server is allocating to the client

10. siaddr - server IP address

11. giaddr - gateway IP address, usually it means the local router

12. chaddr - client hardware address, must be consistent with the hardware type, usually set as the MAC address of the client

13. sname - server name, optional

14. file - legacy BOOTP field, BOOTP servers would send a file meant to boot up the client, as then client wouldn't had a hard drive to boot from

15. options - this is where the magic happens, this field is what makes DHCP more than just a BOOTP extension, here we specify the type of DHCP message, the DNS server's IP etc.

Modules: (Binary Version)

1. `dhcp_json`

This is a very simple file, a JSON file only containing the IP address of the DNS server

2. `dhcp_model.py`

Acts as a simple dataclass, stores all the possible fields of the DHCP header, and is used later for constructing the header with a struct.

3. `dhcp_protocol.py`

Holds all the possible methods of the DORA process (Discover, Offer, Request, Acknowledge), and also methods of packing and unpacking the header's struct.

4. `dhcp_server.py`

The actual DHCP server, sets up the IP pool for allocation and listens on the UDP socket, and when called sets up to work his DORA state machine.

5. `dhcp_service.py`

Acts as the DHCP client, has methods for setting up a DISCOVER and REQUEST DHCP requests.

Modules: (JSON Version)

1. dhcp_protocol.py - this file holds the functions that allows the server and the client to orchestrate the DORA process
2. dhcp_json.json - a very simple file, holds the address of the DNS server, used in the OFFER and ACK responses
3. dhcp_service.py - acts the JSON client, performs the client side of the DORA state machine
4. dhcp_server.py - acts as the JSON server, performs the server side of the DORA process.

The D.O.R.A Process:

The DORA process is the official name for the state machine of the DHCP Protocol.

It acts as the cycle of communication between server and client.

1. DISCOVER:

The client boots up, he doesn't have an IP address yet, can only communicate at Link Level, meaning through his MAC address, the DISCOVER is sent to the FF-FF-FF-FF-FF-FF MAC address, which is configured as a general address which every computer can listen to.

Since the client didn't get an IP yet, he can only communicate with this approach.

```
▼ Dynamic Host Configuration Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0xef60c6fd
  Seconds elapsed: 0
  ▶ Bootp flags: 0x8000, Broadcast flag (Broadcast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: CloudNetwork_e3:a9:d6 (3c:0a:f3:e3:a9:d6)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  ▶ Option: (53) DHCP Message Type (Discover)
  ▶ Option: (255) End
```

2. OFFER

The DHCP Server picks up on the DISCOVER signal and crafts an OFFER response.

The client sent this MAC address in the DISCOVER request, so the server knows where to send his response.

The OFFER consists mainly of one main thing, a free IP address the server can allocate.

```

▼ Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0xef60c6fd
  Seconds elapsed: 0
  ▶ Bootp flags: 0x8000, Broadcast flag (Broadcast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.100.102.100
  Next server IP address: 10.100.102.5
  Relay agent IP address: 0.0.0.0
  Client MAC address: CloudNetwork_e3:a9:d6 (3c:0a:f3:e3:a9:d6)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  ▶ Option: (6) Domain Name Server
  ▶ Option: (53) DHCP Message Type (Offer)
  ▶ Option: (54) DHCP Server Identifier (10.100.102.5)
  ▶ Option: (255) End

```

3. REQUEST:

After the server sent an available IP address in the OFFER, the client now asks permission from the server to use that IP.

This request is sent as checkup - so the server can ensure the client uses the right IP

```

▼ Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0xef60c6fd
  Seconds elapsed: 0
  ▶ Bootp flags: 0x8000, Broadcast flag (Broadcast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: CloudNetwork_e3:a9:d6 (3c:0a:f3:e3:a9:d6)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  ▶ Option: (53) DHCP Message Type (Request)
  ▶ Option: (50) Requested IP Address (10.100.102.100)
  ▶ Option: (54) DHCP Server Identifier (10.100.102.5)
  ▶ Option: (255) End

```

4. ACK:

This is the final step in the state machine, the server now can acknowledge the new client and make sure he can communicate at the local network he's responsible for.

This response is sent so the client can make sure he got permission to use the parameters he got.


```

▼ Dynamic Host Configuration Protocol (ACK)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0xef60c6fd
  Seconds elapsed: 0
  ▶ Bootp flags: 0x8000, Broadcast flag (Broadcast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.100.102.100
  Next server IP address: 10.100.102.5
  Relay agent IP address: 0.0.0.0
  Client MAC address: CloudNetwork_e3:a9:d6 (3c:0a:f3:e3:a9:d6)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  ▶ Option: (6) Domain Name Server
  ▶ Option: (53) DHCP Message Type (ACK)
  ▶ Option: (54) DHCP Server Identifier (10.100.102.5)
  ▶ Option: (255) End

```

Implementation Challenges & Simulations

- A genuine DHCP message is built over a custom Ethernet, IP and UDP headers. Since we are just mimicking a DHCP header, we don't need that.

Instead of setting up a real broadcast through the Ethernet header, we use the UDP broadcast option.

- Our DHCP server holds what called an "IP Pool" - a list of IP addresses that the server can allocate to any client
- Binary vs JSON format - while the binary format we made is more genuine and acts as a "certified copy" of the DHCP protocol, the JSON version is much more easy to understand and to code.
- The DHCP servers we made acts as the entry point of the client, he allows it to communicate in the network, and hands it the DNS IP.

Code Explained

dhcp_model

```

@dataclass
class DHCPpacket:
    op: int = 1          # 1 for Request (Client->Server), 2 for Reply
                        # (Server->Client)
    htype: int = 1       # Ethernet
    hlen: int = 6        # MAC length
    hops: int = 0
    xid: int = 0         # Transaction ID (must match across DORA)
    secs: int = 0        # Lease Time | a time in seconds a client can hold
                        # an IP address
    flags: int = 0x8000  # Broadcast flag (often required for clients
                        # without IPs)
    ciaddr: str = "0.0.0.0" # Client IP address | Redundant in Our Case

```

```

yiaddr: str = "0.0.0.0" # Your IP Address | Client IP Address
siaddr: str = "0.0.0.0" # Server IP Address
giaddr: str = "0.0.0.0"
chaddr: bytes = b'' # Binary MAC address
options: Dict[int, bytes] = field(default_factory=dict)

class DHCPState(bytes, enum.Enum):
    DISCOVER = b'\x01'
    OFFER = b'\x02'
    REQUEST = b'\x03'
    ACK = b'\x05'

```

The main dataclass, acts as an abstraction layer from the binary format to a human readable format
Here we set all the default parameters.

dhcp_protocol

```

def __get_dns() -> str:
    with open(dhcp_config, 'r', encoding='utf-8') as file:
        data = json.load(file)
        return data.get('dns_ip')

def pack_dhcp_packet(packet: DHCPpacket) -> bytes:
    header = struct.pack(
        '!BBBBIHH4s4s4s16s',
        packet.op, packet.htype, packet.hlen, packet.hops,
        packet.xid, packet.secs, packet.flags,
        socket.inet_aton(packet.ciaddr), socket.inet_aton(packet.yiaddr),
        socket.inet_aton(packet.siaddr), socket.inet_aton(packet.giaddr),
        packet.chaddr.ljust(16, b'\x00')
    )
    # 192 bytes of padding (sname + file) + Magic Cookie
    body = header + (b'\x00' * 192) + MAGIC_COOKIE
    for code, val in packet.options.items():
        body += struct.pack('!BB', code, len(val)) + val
    return body + b'\xff'

def unpack_dhcp_packet(data: bytes) -> DHCPpacket:
    # Basic header unpacking (first 44 bytes)
    parts = struct.unpack('!BBBBIHH4s4s4s16s', data[:44])
    pkt = DHCPpacket(
        op=parts[0], xid=parts[4],
        ciaddr=socket.inet_ntoa(parts[7]), yiaddr=socket.inet_ntoa(parts[8]),
        siaddr=socket.inet_ntoa(parts[9]), giaddr=socket.inet_ntoa(parts[10]),
        chaddr=parts[11][:parts[2]]
    )
    # Simple option parser
    offset = 240 # Header(236) + Cookie(4)
    while offset < len(data):
        opt_type = data[offset]
        if opt_type == 255: break

```

```

        length = data[offset+1]
        pkt.options[opt_type] = data[offset+2 : offset+2+length]
        offset += 2 + length
    return pkt

dns_ip = __get_dns()

# Factory functions for the D.O.R.A process

def create_discover(xid, mac_bin):
    p = DHCPpacket(xid=xid, chaddr=mac_bin)
    p.options[53] = DHCPState.DISCOVER
    return pack_dhcp_packet(p)

def create_offer(xid, mac_bin, yiaddr, siaddr):
    p = DHCPpacket(op=2, xid=xid, chaddr=mac_bin, yiaddr=yiaddr, siaddr=siaddr)
    p.options[6] = socket.inet_aton(dns_ip)
    p.options[53] = DHCPState.OFFER
    p.options[54] = socket.inet_aton(siaddr)
    return pack_dhcp_packet(p)

def create_request(xid, mac_bin, requested_ip, siaddr):
    p = DHCPpacket(xid=xid, chaddr=mac_bin)
    p.options[53] = DHCPState.REQUEST
    p.options[50] = socket.inet_aton(requested_ip); p.options[54] =
socket.inet_aton(siaddr)
    return pack_dhcp_packet(p)

def create_ack(xid, mac_bin, yiaddr, siaddr):
    p = DHCPpacket(op=2, xid=xid, chaddr=mac_bin, yiaddr=yiaddr, siaddr=siaddr)
    p.options[6] = socket.inet_aton(dns_ip)
    p.options[53] = DHCPState.ACK
    p.options[54] = socket.inet_aton(siaddr)
    return pack_dhcp_packet(p)

```

Here we define all the DHCP related functions.

Every sub-process in the DORA process is defined here.

We use this function to pack and unpack the header's struct.

dhcp_server

```

import socket
import argparse
import dhcp_protocol
import sys

from dhcp_model import SERVER_PORT, CLIENT_PORT, DHCPState

class DHCPServer:
    def __init__(self, server_ip, interface, subnet_mask="255.255.255.0"):
        self.server_ip = server_ip
        self.interface = interface

```

```

self.subnet = subnet_mask
prefix = ".".join(server_ip.split('.')[:-1])
self.ip_pool = [f"{prefix}.{i}" for i in range(100, 201)]

def handle_packet(self, data):
    pkt = dhcp_protocol.unpack_dhcp_packet(data)
    msg_type = pkt.options.get(53)

    match msg_type:
        case DHCPState.DISCOVER:
            offered_ip = self.ip_pool[0]
            print("[DHCP] OFFER")
            print(f"[DHCP] xid: {pkt.xid} | offered ip: {offered_ip}")
            return dhcp_protocol.create_offer(pkt.xid, pkt.chaddr,
offered_ip, self.server_ip)

        case DHCPState.REQUEST:
            req_ip = socket.inet_ntoa(pkt.options.get(50,
b'\x00\x00\x00\x00'))
            print("[DHCP] ACK")
            print(f"[DHCP] xid: {pkt.xid} | offered ip: {req_ip}")
            return dhcp_protocol.create_ack(pkt.xid, pkt.chaddr, req_ip,
self.server_ip)

    return None

def serve(self, port=SERVER_PORT):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)

    # --- CROSS-PLATFORM INTERFACE LOGIC ---
    if sys.platform.startswith('linux'):
        try:
            # Linux approach: Bind directly to the hardware name
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_BINDTODEVICE,
self.interface.encode())
            print(f"[DHCP] Linux: Bound to interface {self.interface}")
        except PermissionError:
            print("[DHCP] Permission denied: DHCP servers require sudo on
Linux.")

    elif sys.platform == 'win32':
        # Windows approach: Bind to the IP address associated with the
interface
        # On Windows, we bind to the specific local IP to lock the interface
        try:
            # sock.bind((self.server_ip, port))
            sock.bind(('127.0.0.1', port))
            print(f"[DHCP] Windows: Bound to IP {self.server_ip} on interface
{self.interface}")
        except Exception as e:

```

```

        print(f"[DHCP] Windows Bind Error: {e}")
        sock.bind('', port) # Fallback to all interfaces

# Final fallback/default bind for non-Windows or if Linux binding failed
if sys.platform != 'win32':
    sock.bind('', port)

print(f"[DHCP] Server active. IP Serving pool:
{self.ip_pool[0]}-{self.ip_pool[-1]}")
while True:
    data, addr = sock.recvfrom(1024)
    response = self.handle_packet(data)
    if response:
        # sock.sendto(response, ('<broadcast>', CLIENT_PORT))
        sock.sendto(response, ('127.0.0.1', CLIENT_PORT))

if __name__ == "__main__":
    def get_local_ip():
        """Automatically detect the IP of the default interface."""
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            # Doesn't actually connect, just triggers the OS to pick an interface
            s.connect(('8.8.8.8', 80))
            return s.getsockname()[0]
        except Exception:
            return "192.168.1.1" # Total fallback
        finally:
            s.close()

    parser = argparse.ArgumentParser(description="DHCP Server")
    # If the user doesn't provide --ip, get_local_ip() runs automatically
    parser.add_argument("--ip", default=get_local_ip(), help="The Base IP for the
server")
    parser.add_argument("--interface", default="eth0", help="Interface name")

    args = parser.parse_args()
    print(f"[DHCP] Using Base IP: {args.ip}") # Confirms what was chosen
    server = DHCPSTServer(args.ip, args.interface)
    server.serve()

```

This class acts as the main side of the DORA state machine.

Here we can listen to any DHCP clients and generate responses for them.

The server manages an IP pool where he sends IP addresses to the clients.

dhcp_service

```

import socket
import uuid
import random
from dhcp_bitwise import dhcp_protocol

```

```

from dhcp_bitwise.dhcp_model import SERVER_PORT, CLIENT_PORT, DHCPState

class DHCPService:
    def __init__(self):
        self.xid = random.randint(0, 0xFFFFFFFF)
        self.mac_addr = self._get_mac_binary()

    @staticmethod
    def _get_mac_binary() -> bytes:
        node = uuid.getnode()
        return node.to_bytes(6, 'big')

    def get_ip_and_dns(self):
        """Executes DORA and returns (assigned_ip, dns_server)"""
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
            sock.settimeout(5.0)

            sock.bind(('', CLIENT_PORT))
            server_addr = ('127.0.0.1', SERVER_PORT) # Localhost for testing

            try:
                # DISCOVER
                discover_data = dhcp_protocol.create_discover(self.xid,
self.mac_addr)
                sock.sendto(discover_data, server_addr)

                # OFFER
                data, _ = sock.recvfrom(1024)
                offer_pkt = dhcp_protocol.unpack_dhcp_packet(data)

                # REQUEST
                request_data = dhcp_protocol.create_request(
                    self.xid, self.mac_addr, offer_pkt.yiaddr, offer_pkt.siaddr
                )
                sock.sendto(request_data, server_addr)

                # ACK
                data, _ = sock.recvfrom(1024)
                ack_pkt = dhcp_protocol.unpack_dhcp_packet(data)

                if ack_pkt.options.get(53) == DHCPState.ACK:
                    dns_bytes = ack_pkt.options.get(6)
                    dns_str = socket.inet_ntoa(dns_bytes) if dns_bytes else None
                    return ack_pkt.yiaddr, dns_str

            except socket.timeout:
                print("[-] DHCP Timeout.")
            return None, None

```

This class is the other side of the DORA state machine.

Here a client can send a request to get a new IP, here we're generating requests by the RFC.

dhcp_protocol (JSON)

```
import json
import socket

class DHCPServer:
    def __init__(self, ip_mask="10.100.102", allocation=10, dns=None):
        self.ip_mask = ip_mask
        self.allocation = max(2, min(allocation, 254)) # Clamp between 2 and 254
        self.dns_server = dns or self._load_dns_from_file()
        self.port = 6767

        # IP Pool Management
        # We'll start allocating from .10 up to .10 + allocation
        self.start_ip = 10
        self.pool = {} # Dictionary to track {transaction_id: assigned_ip}

    @staticmethod
    def _load_dns_from_file():
        try:
            with open("dhcp.json", "r") as f:
                data = json.load(f)
                return data.get("dns_server", "10.100.102.5")
        except (FileNotFoundError, json.JSONDecodeError):
            return "10.100.102.5"

    def _get_next_available_ip(self, xid):
        """
        Simple pool logic: find the next free index in our allocation range.
        """
        # If this transaction ID already has an assignment, return it
        if xid in self.pool:
            return self.pool[xid]

        # Otherwise, find a new one
        for i in range(self.start_ip, self.start_ip + self.allocation):
            potential_ip = f"{self.ip_mask}.{i}"
            if potential_ip not in self.pool.values():
                self.pool[xid] = potential_ip
                return potential_ip

        return None # Pool exhausted

    def handle(self, raw_data):
        if not raw_data:
            return b''
        try:
            data = json.loads(raw_data.decode("utf8"))
```

```

msg_type = data.get("message_type")
xid = data.get("transaction_id")

print(f"[DHCP RECEIVE] {msg_type} | ID: {xid}")

# Allocate or retrieve IP from the pool
assigned_ip = self._get_next_available_ip(xid)
if not assigned_ip:
    print(f"[DHCP ERROR] Pool exhausted! Cannot serve ID: {xid}")
    return json.dumps({"error": "No IPs available"}).encode("utf8")

out_type = "OFFER" if msg_type == "DISCOVER" else "ACK"

response = {
    "message_type": out_type,
    "transaction_id": xid,
    "ip_address": assigned_ip,
    "dns_server": self.dns_server
}

final_json = json.dumps(response)
print(f"[DHCP SEND] {out_type} | Assigned: {assigned_ip} | DNS: {self.dns_server}")
return final_json.encode("utf8")

except Exception as e:
    print(f"[DHCP ERROR] {e}")
    return b''

def serve(self):
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        try:
            sock.bind(("0.0.0.0", self.port))
            print(f"\n[DHCP] DHCP Server Live | Port: {self.port}")
            #print(f"[DHCP] Pool: {self.ip_mask}.{self.start_ip} to {self.start_ip + self.allocation - 1}")
            print(f"[DHCP] Pool: {self.ip_mask}.{self.start_ip} to {self.ip_mask}.{str(self.allocation - 1)}")
            print("-" * 50)
        except OSError as e:
            print(f"[DHCP] Bind Error: {e}")
            return

        while True:
            raw_data, addr = sock.recvfrom(1024)
            response = self.handle(raw_data)
            if response:
                sock.sendto(response, addr)

```

Here we define all the core functions that will be used by the server

dhcp_server (JSON)


```

"""
dhcp_server.py - CLI entrypoint for DHCP Server

Usage:
    python dhcp_server.py --ip-mask <mask> --allocation <count> --dns <dns_ip>

Example:
    python dhcp_server.py --ip-mask 192.168.1 --allocation 50 --dns 10.100.102.5
"""

```

```

import argparse
from dhcp_protocol import DHCP Server

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Start a DHCP server.",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    parser.add_argument(
        "--ip-mask",
        type=str,
        default="10.100.102",
        help="24-bit IP prefix for the address pool (e.g. '192.168.1').",
    )
    parser.add_argument(
        "--allocation",
        type=int,
        default=10,
        help="Number of IPs to allocate (2-256).",
    )
    # Added the --dns argument to support your static DNS requirement
    parser.add_argument(
        "--dns",
        type=str,
        default=None,
        help="Static DNS IP to provide to clients (overrides dhcp.json if provided).",
    )
    return parser.parse_args()

def main():
    args = parse_args()

    # The server initialization now passes all three parameters to the DHCP Server
    class
    server = DHCP Server(
        ip_mask=args.ip_mask,
        allocation=args.allocation,
        dns=args.dns
    )

```

```

print(
    f"[DHCP] Starting server | "
    f"mask={args.ip_mask}.0/24 | "
    f"pool={args.allocation} IPs | "
    f"DNS={server.dns_server}"
)

# Start the infinite network loop
try:
    server.serve()
except KeyboardInterrupt:
    print("\n[DHCP] Server shutting down manually.")

if __name__ == "__main__":
    main()

```

Here we set up the server

dhcp_service (JSON)

```

import socket
import json
import random

class DHCPService:
    def __init__(self):
        self.xid = random.randint(1000, 9999)
        self.port_server = 6767 # Matches Port_In in dhcp_protocol.py
        self.port_client = 6868 # Matches Port_Out in dhcp_protocol.py

    def get_ip_and_dns(self):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
            sock.settimeout(5.0)
            sock.bind(('', self.port_client))

            # DISCOVER
            discover = {"message_type": "DISCOVER", "transaction_id": self.xid}
            sock.sendto(json.dumps(discover).encode(), ('255.255.255.255',
self.port_server))

            try:
                # OFFER
                data, _ = sock.recvfrom(1024)
                offer = json.loads(data.decode())
                offered_ip = offer.get("ip_address")

                # REQUEST
                request = {
                    "message_type": "REQUEST",

```

```

        "transaction_id": self.xid,
        "requested_ip": offered_ip
    }
    sock.sendto(json.dumps(request).encode(), ('255.255.255.255',
self.port_server))

    # ACK
    data, _ = sock.recvfrom(1024)
    ack = json.loads(data.decode())
    return ack.get("ip_address"), ack.get("dns_server")

except (socket.timeout, json.JSONDecodeError):
    return None, None

```

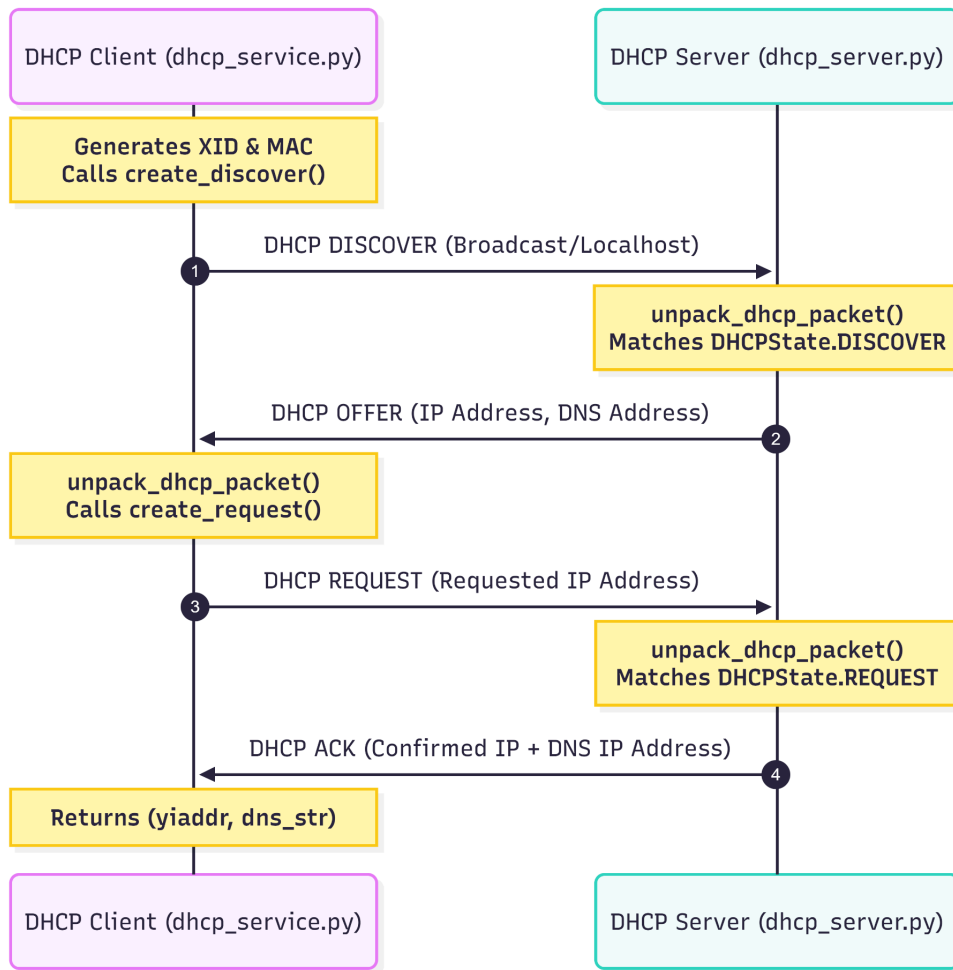
Here we define all the core functions of the client

Summary of Prompts Used During Development - DHCP:

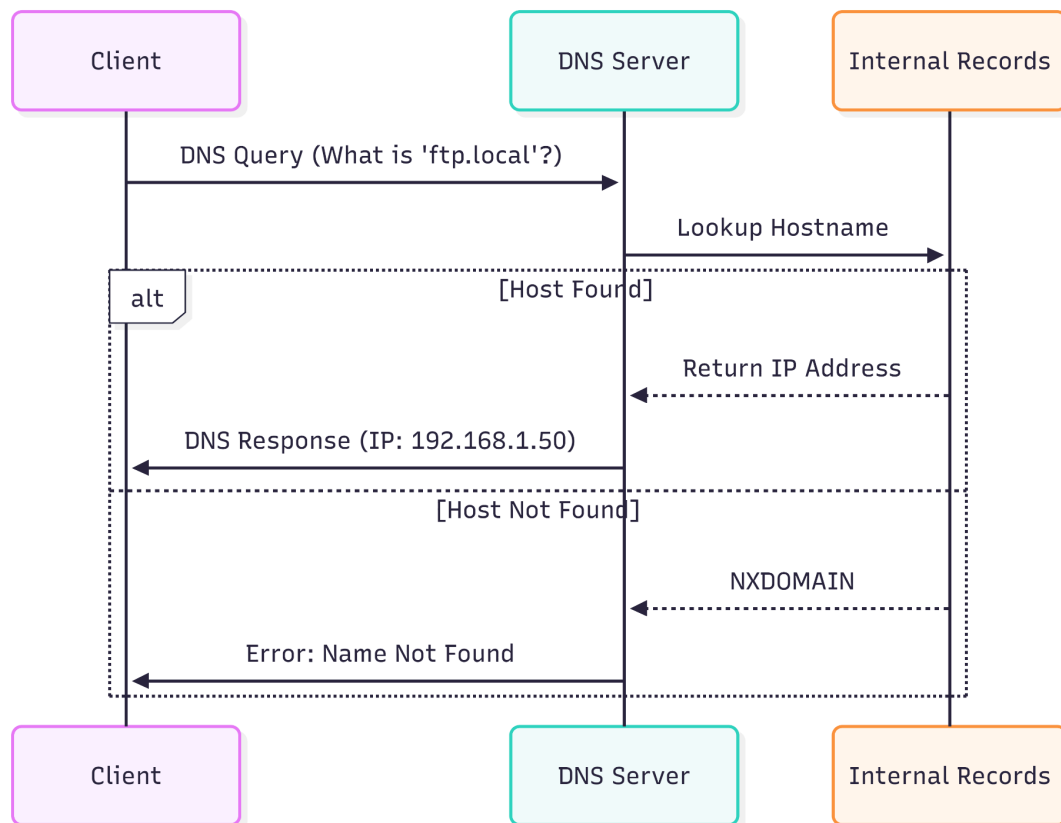
- You're a senior networking engineer, explain about the DHCP protocol
- You're a senior networking engineer, explain about the DORA process
- You're a senior networking engineer, explain the DHCP header
- You're a senior networking engineer. I'm building a DHCP server and noticed that the options field has numbers, like 6 or 54, care to explain?
- You're a senior networking engineer. I'm trying to figure out how the DHCP protocol works but the RFC article has too much information around the protocol so its hard to find the actual core of it. please read it and explain to me the essence of the protocol: how it works, what every field in its header mean and what are the possible values (only the most relevant ones) and how the server should behave based on each response and vice versa
- You're a senior networking engineer. Since DHCP is working over ethernet packets, then we cant use UDP sockets, then what type of socket should we use?

Workflow

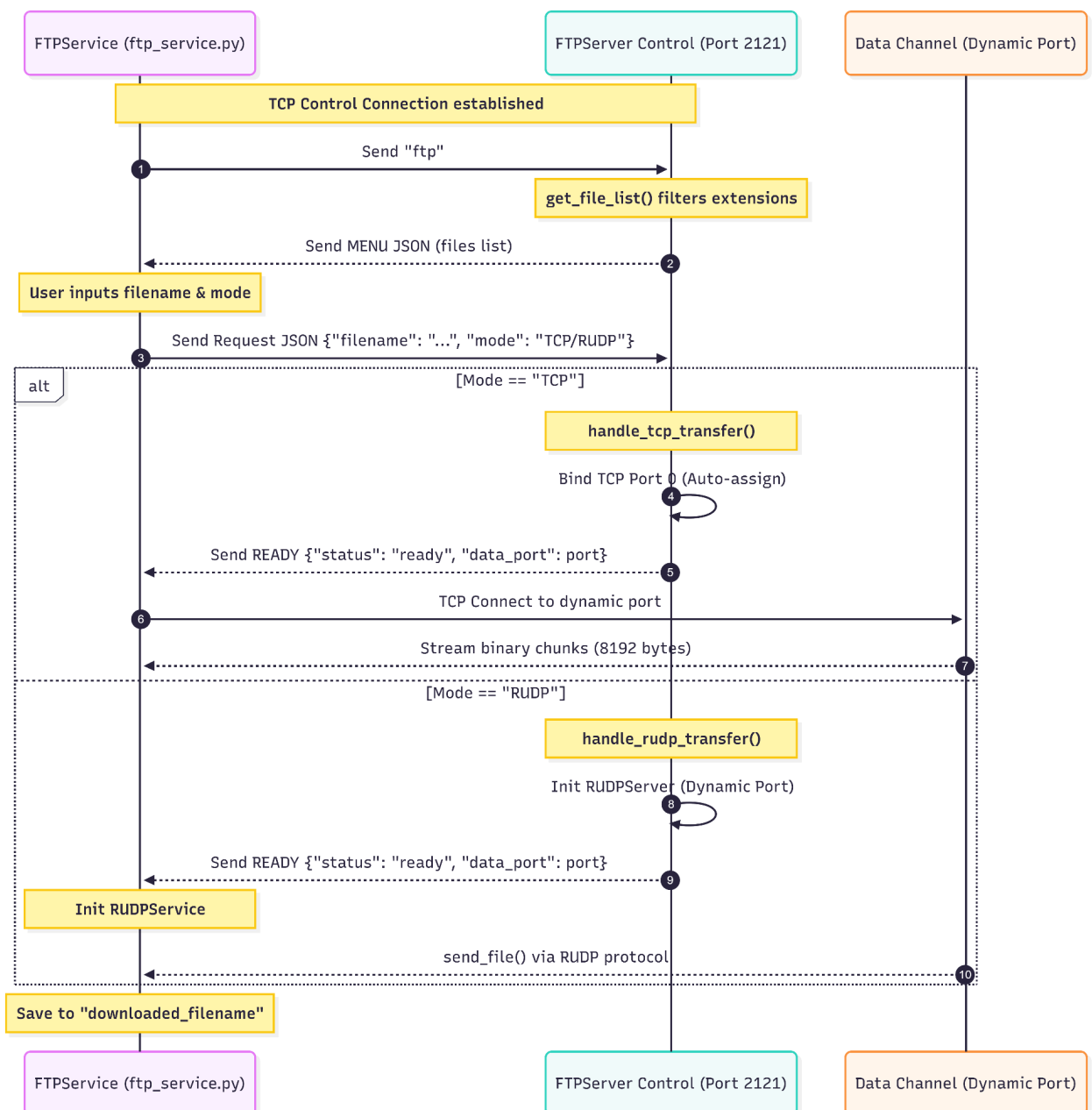
DHCP



DNS



FTP



Dissectors

Since we partially used JSON based protocols, we decided to write dissectors for them. A dissector is a Lua script that acts as a bridge from the actual stream to a wireshark frame.

DHCP_JSON

```
local dhcp_json_proto = Proto("dhcp_json", "JSON DHCP Protocol")

-- Standalone Fields definition
local f_msg_type = ProtoField.string("dhcp_json.type", "Message Type")
local f_xid      = ProtoField.string("dhcp_json.xid", "Transaction ID")
local f_ip       = ProtoField.string("dhcp_json.ip", "Assigned IP Address")
local f_dns      = ProtoField.string("dhcp_json.dns", "DNS Server IP")

-- Registering fields (Removed f_payload here)
dhcp_json_proto.fields = { f_msg_type, f_xid, f_ip, f_dns }

function dhcp_json_proto.dissector(buffer, pinfo, tree)
    pinfo.cols.protocol = "JSON_DHCP"

    -- Convert buffer to string for pattern matching
    local raw_data = buffer():string()

    -- Create the main tree item
    local subtree = tree:add(dhcp_json_proto, buffer(), "JSON DHCP Protocol Data")

    -- 1. Extract values using pattern matching
    local msg_type = string.match(raw_data, '"message_type"%s*:%s*"(.-)"')
    local xid      = string.match(raw_data, '"transaction_id"%s*:%s*"([^\s,}]+)')
    local ip       = string.match(raw_data, '"ip_address"%s*:%s*"(.-)"') or
string.match(raw_data, '"requested_ip"%s*:%s*"(.-)"')
    local dns      = string.match(raw_data, '"dns_server"%s*:%s*"(.-)"')

    -- 2. Add fields to tree as Standalones (No raw JSON blob)
    if msg_type then
        subtree:add(f_msg_type, msg_type)
    end

    if xid then
        subtree:add(f_xid, xid)
    end

    if ip then
        subtree:add(f_ip, ip)
    end
end
```

```

if dns then
    subtree:add(f_dns, dns)
end

-- 3. Update the Wireshark Info Column for quick scanning
pinfo.cols.info = "DHCP " .. (msg_type or "Unknown") .. " [ID: " .. (xid or
"?") .. "]"
end

-- Map to your custom ports
local udp_port = DissectorTable.get("udp.port")
udp_port:add(6767, dhcp_json_proto)
udp_port:add(6868, dhcp_json_proto)

```

DNS_JSON

```

local dns_json_proto = Proto("dns_json", "JSON DNS Protocol")

-- Standalone Fields definition
local f_url = ProtoField.string("dns_json.url", "Queried URL")
local f_resp_ip = ProtoField.string("dns_json.resolved_ip", "Resolved IP")

-- Registering standalone fields only
dns_json_proto.fields = { f_url, f_resp_ip }

function dns_json_proto.dissector(buffer, pinfo, tree)
    pinfo.cols.protocol = "JSON_DNS"
    local raw_data = buffer():string()

    -- Create the main tree item
    local subtree = tree:add(dns_json_proto, buffer(), "JSON DNS Data")

    -- 1. Extract values via pattern matching
    local url = string.match(raw_data, '"url"%s*:%s*"(.-)"')
    local ip = string.match(raw_data, '"ip"%s*:%s*"(.-)"')

    -- 2. Add fields to tree ONLY if they exist in the JSON
    if url then
        subtree:add(f_url, url)
        pinfo.cols.info = "DNS Query: " .. url
    end

    if ip then
        subtree:add(f_resp_ip, ip)
        pinfo.cols.info = "DNS Response: " .. ip
    end

    -- 3. Raw JSON is no longer added to the tree
end

```



```

local udp_port = DissectorTable.get("udp.port")
udp_port:add(9000, dns_json_proto)
-- If you're testing on mDNS port, uncomment the line below:
-- udp_port:add(5353, dns_json_proto)

```

FTP (FOJP)

FOJP (File Transfer Over JSON Protocol) is a JSON based protocol which mimics the FTP state machine.

It sends the same requests and responses, but in JSON rather than plain text

```

-- Define the Protocol
local fojp_proto = Proto("FOJP", "File Transfer Over JSON")

-- Field Definitions
local f_type      = ProtoField.string("fojp.type", "Message Type")
local f_files     = ProtoField.string("fojp.files", "Available Files")
local f_filename  = ProtoField.string("fojp.filename", "Requested File")
local f_mode      = ProtoField.string("fojp.mode", "Transfer Mode")
local f_status    = ProtoField.string("fojp.status", "Status")
local f_data_port = ProtoField.uint16("fojp.data_port", "Dynamic Data Port")
local f_ack_num   = ProtoField.int32("fojp.ack_num", "RUDP ACK Number")

fojp_proto.fields = { f_type, f_files, f_filename, f_mode, f_status,
f_data_port, f_ack_num }

-- Helper to extract JSON values
local function get_json_val(str, key)
    local pattern = '"' .. key .. '"%s*:%s*"?(["%,%s%}]]*)"'
    return str:match(pattern)
end

function fojp_proto.dissector(buffer, pinfo, tree)
    local length = buffer:len()
    if length == 0 then return end

    local raw_data = buffer(0, length):string()

    if not (raw_data:match("^%s*{") or raw_data:match("}$")) then return end

    pinfo.cols.protocol = "FOJP"
    local subtree = tree:add(fojp_proto, buffer(), "FOJP Control Protocol")

    local m_type = get_json_val(raw_data, "type")
    local status = get_json_val(raw_data, "status")
    local d_port = get_json_val(raw_data, "data_port")
    local fname  = get_json_val(raw_data, "filename")
    local mode   = get_json_val(raw_data, "mode")

```

```

-- --- Logic for Control Channel ---

if m_type == "MENU" then
    subtree:add(f_type, "MENU")
    -- Optimized File List Extraction
    local files = raw_data:match('"files"%s*:%s*%[[^%]]*)%')
    if files then
        local cleaned_files = files:gsub("'", ''):gsub('%s+', '')
        -- Pass cleaned_files as the third argument to display it in the tree
label
        subtree:add(f_files, buffer(), cleaned_files)
    end
    pinfo.cols.info = "Sending File Menu"

elseif fname then
    subtree:add(f_filename, buffer(), fname)
    if mode then subtree:add(f_mode, buffer(), mode) end
    pinfo.cols.info = "Request [" .. fname .. "] via " .. (mode or "UNKNOWN")

elseif status then
    subtree:add(f_status, buffer(), status)
    if d_port then
        subtree:add(f_data_port, buffer(), tonumber(d_port))
        pinfo.cols.info = "READY on Port " .. d_port
    else
        pinfo.cols.info = "Status -> " .. status
    end
end

elseif m_type == "ACK" then
    local ack = get_json_val(raw_data, "num")
    subtree:add(f_type, "RUDP_ACK")
    if ack then
        subtree:add(f_ack_num, buffer(), tonumber(ack))
        pinfo.cols.info = "RUDP: ACK #" .. ack
    end
end
end
end

local tcp_port = DissectorTable.get("tcp.port")
tcp_port:add(2121, fojproto)

```

CRUDP

CRUDP (Custom Reliable User Datagram Protocol) is the name of the RUDP protocol we wrote

```

-- CRUDP Protocol Dissector (Binary Version)
local crudp_proto = Proto("CRUDP", "Custom Reliable UDP")

```

```

-- Header Fields
local f_seq_num = ProtoField.uint32("crudp.seq_num", "Sequence Number",
base.DEC)
local f_total_p = ProtoField.uint32("crudp.total_packets", "Total Packets",
base.DEC)
local f_flags    = ProtoField.uint8("crudp.flags", "Flags", base.HEX)

-- Bitmask Flags
local f_flag_syn = ProtoField.bool("crudp.flags.syn", "SYN (Connection Init)",
8, nil, 0x04)
local f_flag_ack = ProtoField.bool("crudp.flags.ack", "ACK (Acknowledgment)", 8,
nil, 0x02)
local f_flag_fin = ProtoField.bool("crudp.flags.fin", "FIN (Teardown)", 8, nil,
0x01)

-- Padding and Meta
local f_reserved = ProtoField.bytes("crudp.reserved", "Padding (3 bytes)",
base.NONE)
local f_pkt_size = ProtoField.uint32("crudp.size", "Total Frame Size", base.DEC)

crudp_proto.fields = { f_seq_num, f_total_p, f_flags, f_flag_syn, f_flag_ack,
f_flag_fin, f_reserved, f_pkt_size }

function crudp_proto.dissector(buffer, pinfo, tree)
    local length = buffer:len()
    if length < 12 then return end

    pinfo.cols.protocol = "CRUDP"

    local seq_num = buffer(0, 4):uint()
    local total_p = buffer(4, 4):uint()
    local flags    = buffer(8, 1):uint()

    -- Determine Packet Type
    local p_type = "DATA"
    if bit.band(flags, 0x04) ~= 0 then p_type = "SYN"
    elseif bit.band(flags, 0x02) ~= 0 then p_type = "ACK"
    elseif bit.band(flags, 0x01) ~= 0 then p_type = "FIN"
    end

    -- Update Info Column
    if p_type == "DATA" then
        pinfo.cols.info = string.format("DATA | Seq: %d | Total: %d", seq_num,
total_p)
    else
        pinfo.cols.info = string.format("%s | Num: %d", p_type, seq_num)
    end

    -- Main Header Tree
    local subtree = tree:add(crudp_proto, buffer(0, 12), "CRUDP Header (" ..
p_type .. ")")
    subtree:add(f_seq_num, buffer(0, 4))

```

```

subtree:add(f_total_p, buffer(4, 4))

-- Flags Subtree
local flag_tree = subtree:add(f_flags, buffer(8, 1))

-- Logical Data Display
if flags == 0 then
    flag_tree:add(buffer(8,1), ".... ...0 = Data: True"):set_generated()
else
    flag_tree:add(buffer(8,1), ".... ...0 = Data: False"):set_generated()
end

flag_tree:add(f_flag_syn, buffer(8, 1))
flag_tree:add(f_flag_ack, buffer(8, 1))
flag_tree:add(f_flag_fin, buffer(8, 1))

-- Padding and Size
subtree:add(f_reserved, buffer(9, 3))
subtree:add(f_pkt_size, length):set_generated()

-- Payload Section
if length > 12 then
    local payload_len = length - 12
    tree:add(buffer(12, payload_len), "Payload Data (" .. payload_len .. "
bytes)")
end
end

local udp_port = DissectorTable.get("udp.port")
udp_port:add(56185, crudp_proto)

```

Wireshark Flows

RFC

```

PS C:\Users\Mor\PycharmProjects\Networking-Project\RFCClient> python client.py
Bound to: 10.100.102.100 | DNS: 127.0.0.1
--- Resolving ftp.project_gay.com ---
binding to 127.0.0.4

Welcome, for quitting - press QUIT else press FTP:

```

dhcp						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	DHCP	276	DHCP Discover - Transaction ID 0x36d4083e
2	0.000511100	127.0.0.1	127.0.0.1	DHCP	288	DHCP Offer - Transaction ID 0x36d4083e
3	0.000694000	127.0.0.1	127.0.0.1	DHCP	288	DHCP Request - Transaction ID 0x36d4083e
4	0.000905300	127.0.0.1	127.0.0.1	DHCP	288	DHCP ACK - Transaction ID 0x36d4083e

dns						
No.	Time	Source	Destination	Protocol	Length	Info
5	0.002402900	127.0.0.1	127.0.0.1	DNS	69	Standard query 0x1234 A ftp.project_gay.com
6	0.003018300	127.0.0.1	127.0.0.1	DNS	85	Standard query response 0x1234 A ftp.project_gay.com A 127.0.0.4

```
PS C:\Users\Mor\PycharmProjects\Networking-Project\RFCClient> python client.py
Bound to: 10.100.102.100 | DNS: 127.0.0.1
--- Resolving ftp.project_gay.com ---
binding to 127.0.0.4
```

```
Welcome, for quitting - press QUIT else press FTP: ftp
1. Dead Inside.mp3
2. Jimmy.mp3
3. Skeletons.gif
4. Skulls.jpg
5. test.gif
6. TOOL-Jimmy 10-16-1996 HD.mp4
```

```
Select file number: █
```

```
Bound to: 10.100.102.100 | DNS: 127.0.0.1
--- Resolving ftp.project_gay.com ---
binding to 127.0.0.4
```

```
Welcome, for quitting - press QUIT else press FTP: ftp
1. Dead Inside.mp3
2. Jimmy.mp3
3. Skeletons.gif
4. Skulls.jpg
5. test.gif
6. TOOL-Jimmy 10-16-1996 HD.mp4
```

```
Select file number: 1
Select mode (TCP/RUDP): rudp
```

```
Welcome, for quitting - press QUIT else press FTP: ftp
1. Dead Inside.mp3
2. Jimmy.mp3
3. Skeletons.gif
4. Skulls.jpg
5. test.gif
6. TOOL-Jimmy 10-16-1996 HD.mp4
```

```
Select file number: 1
Select mode (TCP/RUDP): rudp
[RUDP] Sent SYN to ('127.0.0.4', 51213), waiting for data...
RUDP Progress: 1430/2069 █
```

No.	Time	Source	Destination	Protocol	Length	Info
27	132.225371600	127.0.0.1	127.0.0.4	CRUDP	44	SYN Num: 0
28	132.227216100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 0 Total: 2069
29	132.227309400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 1 Total: 2069
30	132.227321100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 2 Total: 2069
31	132.227330200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 3 Total: 2069
32	132.227338900	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 4 Total: 2069
33	132.227347600	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 5 Total: 2069
34	132.227356400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 6 Total: 2069
35	132.227365000	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 7 Total: 2069
36	132.227375500	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 8 Total: 2069
37	132.227386800	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 9 Total: 2069
38	132.227384600	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 0
39	132.227553800	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 1
40	132.227623700	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 2
41	132.227820800	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 3
42	132.227889900	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 4
43	132.228100400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 5
44	132.228301500	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 6
45	132.228482100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 7
46	132.228526700	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 8
47	132.228572300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 9
48	132.232811800	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 10 Total: 2069
49	132.232872400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 11 Total: 2069
50	132.232892200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 12 Total: 2069
51	132.232908100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 13 Total: 2069
52	132.232923700	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 14 Total: 2069
53	132.232975400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 15 Total: 2069
54	132.232998200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 16 Total: 2069
55	132.233008100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 17 Total: 2069
56	132.233022600	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 18 Total: 2069
57	132.233031200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 19 Total: 2069
58	132.233062000	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 10
59	132.233586600	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 11
60	132.233929900	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 12
61	132.234216200	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 13
62	132.234551600	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 14
63	132.234521900	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 15
64	132.234687700	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 16
65	132.234792300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 17
66	132.234843800	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 18
67	132.234879400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 19
68	132.234868100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 20 Total: 2069

JSON

```
PS C:\Users\Mor\PycharmProjects\Networking-Project\JSONClient> python client.py
[+] Bound to: 10.100.102.10 | DNS Server: 127.0.0.1
[+] Found FTP Server at: 127.0.0.4
```

Welcome, for quitting - press QUIT else press FTP: ftp

No.	Time	Source	Destination	Protocol	Length	Info
5	63.656005000	10.100.102.5	255.255.255.255	JSON_DHCP	84	DHCP DISCOVER [ID: 4014]
6	63.656632400	10.100.102.5	10.100.102.5	JSON_DHCP	139	DHCP OFFER [ID: 4014]
7	63.656871100	10.100.102.5	255.255.255.255	JSON_DHCP	116	DHCP REQUEST [ID: 4014]
8	63.657113800	10.100.102.5	10.100.102.5	JSON_DHCP	137	DHCP ACK [ID: 4014]

No.	Time	Source	Destination	Protocol	Length	Info
9	63.658048100	127.0.0.1	127.0.0.1	JSON_DNS	58	DNS Query: ftp.example.com
10	63.658404200	127.0.0.1	127.0.0.1	JSON_DNS	51	DNS Response: 127.0.0.4

```
Welcome, for quitting - press QUIT else press FTP: ftp
1. Dead Inside.mp3
2. Jimmy.mp3
3. Skeletons.gif
4. Skulls.jpg
5. test.gif
6. TOOL-Jimmy 10-16-1996 HD.mp4
```



```

Select file number: 1
Select mode (TCP/RUDP): rudp
[RUDP] Sent SYN to ('127.0.0.4', 64164), waiting for data...
RUDP Progress: 2069/2069
[RUDP] FIN received. Closing.

```

RUDP Download Complete: downloaded_Dead Inside.mp3

Welcome, for quitting - press QUIT else press FTP:

fojp						
No.	Time	Source	Destination	Protocol	Length	Info
16	71.311516500	127.0.0.4	127.0.0.1	FOJP	179	Sending File Menu
18	75.797661000	127.0.0.1	127.0.0.4	FOJP	91	Request [Dead] via RUDP
20	75.799113500	127.0.0.4	127.0.0.1	FOJP	99	READY on Port 64164

crudp						
No.	Time	Source	Destination	Protocol	Length	Info
22	75.800080200	127.0.0.1	127.0.0.4	CRUDP	44	SYN Num: 0
23	75.803316800	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 0 Total: 2069
24	75.803697000	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 1 Total: 2069
25	75.803723400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 2 Total: 2069
26	75.803733500	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 3 Total: 2069
27	75.803743800	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 4 Total: 2069
28	75.803753800	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 5 Total: 2069
29	75.803763200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 6 Total: 2069
30	75.803777500	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 7 Total: 2069
31	75.803787400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 8 Total: 2069
32	75.803797400	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 9 Total: 2069
33	75.804117100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 0
34	75.804562500	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 1
35	75.805155300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 2
36	75.805628000	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 3
37	75.806390100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 4
38	75.806871300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 5
39	75.807566300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 6
40	75.807813100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 7
41	75.808299300	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 8
42	75.808583800	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 9
43	75.809319100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 10 Total: 2069
44	75.809354900	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 11 Total: 2069
45	75.809369600	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 12 Total: 2069
46	75.809381900	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 13 Total: 2069
47	75.809392200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 14 Total: 2069
48	75.809403300	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 15 Total: 2069
49	75.809413900	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 16 Total: 2069
50	75.809430100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 17 Total: 2069
51	75.809440000	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 18 Total: 2069
52	75.809451000	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 19 Total: 2069
53	75.809830100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 10
54	75.810236500	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 11
55	75.810676100	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 12
56	75.810850400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 13
57	75.811160400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 14
58	75.811287400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 15
59	75.811818500	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 16
60	75.812120400	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 17
61	75.812323000	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 18
62	75.812766200	127.0.0.1	127.0.0.1	CRUDP	44	ACK Num: 19
63	75.814834300	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 20 Total: 2069
64	75.814864100	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 21 Total: 2069
65	75.814874200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 22 Total: 2069
66	75.814883200	127.0.0.1	127.0.0.1	CRUDP	1494	DATA Seq: 23 Total: 2069

Q & A

TCP vs QUIC

Four differences between TCP and QUIC:

1. TCP, as a transport layer protocol, requires a 3 way handshake to start a connection, the famous SYN => SYN/ACK => ACK.
however, QUIC runs over TLS & UDP that does not utilize a 3 way handshake, rather a more

sophisticated approach.

QUIC uses TLS under the hood for data security and uses its handshake for sending its own handshake data over, meaning - since TLS uses a 2 way handshake (Client Hello => Server Hello) we are sending the QUIC handshake in less time than

TCP.

2. TCP is a kernel space protocol, meaning - the Operating System is responsible for using it, if a developer wishes to add new features to TCP he would have to write it directly to the kernel, and force himself (and basically everyone else on the planet) to update their OS, QUIC on the other hand runs in user space - over UDP.
meaning - we can update it without ever needing to write to the kernel, or force anyone to update their OS, or even tell anyone.
all we gotta do is send a new update to Google Chrome
(and any other browser that uses QUIC), the users just need to update their browser.

3. both TCP and QUIC are a form of a RDT protocol (Reliable Data Transfer), meaning they both can detect packet loss and retransmit them, but there's a major difference in how that mechanism is implemented.
TCP is a classic RDT protocol, I would say even a "dumb" one.
when a TCP Client/Server detects a packet loss, it would send a double ACK signal for the last packet in order that got transmitted before, signaling to the other side that a packet was not transmitted, therefore - until the lost packet wouldn't get retransmitted, the entire stream gets blocked. This is often called the "Head of Line Blocking" problem.
QUIC solves this problem by using those mechanism:

1. Independent Streams:
QUIC breaks its payload into multiple streams, each containing a different kind of information, this way if one stream is lost it wouldn't affect the whole session, only the specific information from that packet will get affected, and will get retransmitted in the next round.
2. chronological packet ordering:
TCP knows how many packets were transmitted by the calculation of the ACK it receives.
if the server has sent a packet and hasn't received an ACK on time, it will send it again.
now, the sender got an ACK but it cannot know if it was the first packet, which just got delayed, or it's meant for the retransmitted packet.

It's crucial for determining the RTT correctly.

QUIC solves this problem by using chronological naming convention.

each packet is assigned a number, and if one packet gets lost and needs to get retransmitted, it will get a new number instead of the old one.

this way once the sender receives an ACK, it can know for sure if the ACK was delayed or meant for the retransmitted packet.

4. Both TCP and QUIC are connection oriented protocols, meaning they require some form of digital handshake between the server and the client to ensure connectivity before starting the session.

however, TCP - being a stateful protocol is designed to force quit when the client or the server change their network (from one WiFi network to another for example).

QUIC solves this problem by using what is called a "Connection ID" as identification.

TCP uses the IP addresses and ports, parameters that might change when a connection is suddenly stopped.

This forces the TCP Client and Server to reconnect and handshake again.

QUIC guarantees this won't happen by using the Connection ID as a session identifier.

While IPs and Ports can change, Connection ID is given to the client by the QUIC Server himself.

Vegas vs CUBIC

TCP had some variants over the years, each was designed to tackle the problems with a different approach.

The main objective is always minimizing packet loss over time.

While some try to detect packet loss as an approach, some decide to try and predict when such loss might happen.

TCP Vegas is a variant of the TCP family, its main objective is to predict when a loss might occur, and handle it effectively.

Meaning - its delay oriented.

TCP CUBIC on the other hand is what we called "loss based".

CUBIC works by following the moment when we experience a loss and the window gets cut by half and ramp up the congestion window back to the last point where its maximized as fast as possible.

Mechanisms:

TCP Vegas:

The main mechanism is detecting packet loss by tracking the Round Trip Time between a packet and its ACK signal, and comparing that time to the expected RTT.

if the actual RTT is less than or equal to the expected - then there is no loss of bandwidth, we can continue filling up the window.

Otherwise - the RTT exceeds the expected RTT by some setted threshold, then it would slow down the rate before a loss might happen.

CUBIC:

When CUBIC detects a loss, it cuts the transmit rate by half and goes back quickly to the rate at which packets were transmitted without a loss the last time.

When the window finally reaches this rate it starts ramping it up more “aggressively”.

TCP CUBIC uses a function of the difference between the current time and the future time when its window will be full again.

If we let the current time be T , and the future time be K ,

Then the function will look something like:

$$W(t) = (t - K)^3$$

When the functions is shifted with some constants

What is the BGP Protocol?

BGP (Border Gateway Protocol) is an Application Layer Protocol, used for determining transmission routes for routers.

Each router is called an “Autonomous System” or AS for short.

BGP allows each AS to exchange its inner routes parameters (like subnet mask) to the outer worlds, effectively allowing other routers to transmit packets through it.

Basically - BGP is a standard EGP protocol (Exterior Gateway Protocol) - Allowing for router to router communication.

How is BGP Different From OSPF?

OSPF (Open Shortest Path First) is also a routing protocol.

But the main difference from BGP is that while BGP is used between networks, OSPF is only used in a single network.

The main purpose of OSPF as its name suggests, is to find the shortest path (route) to a specific endpoint, using the Dijkstra algorithm.

BGP doesn't necessarily take the shortest path, it takes the path most suitable within the moment.

Applicatio n	Port Src	Port Dest	IP Src	IP Dest	Mac Src	Mac Dest
DHCP	6868	6767	127.0.0.1	127.0.0.1	3C-0A-F3-E 3-A9-D6	00-00-00-0 0-00-00
DNS	8053	62786	127.0.0.1	127.0.0.1	00-00-00-0 0-00-00	00-00-00-0 0-00-00

FTP	0 (randomly selected)	2121	127.0.0.1	127.0.0.1	00-00-00-0 0-00-00	00-00-00-0 0-00-00
-----	-----------------------------	------	-----------	-----------	-----------------------	-----------------------

In case we introduce a NAT between the client and the servers, a few things can happen:
 Since the client needs to communicate with the DHCP at the link level the process can break entirely.
 A NAT might strip the Ethernet header, and make the entire packet obsolete.

About DNS.

The NAT acts as a bridge between the DNS's IP and the client's IP, effectively creating a delay.
 Therefore - the IP the DNS sees would change to the NAT's IP, and the port would also change.

About FTP.

Since we are using a custom protocol rather than the real FTP.
 The situation here is different.

We would have to port forward from the server to the NAT and from the NAT to the client.
 The IP addresses and ports would change to the IP of the NAT and to the port we used in the NAT.

Differences Between ARP and DNS

We all know that bits and bytes are not readable for us humans.
 So if we want to understand what is happening on the web we are going to need a translator.
 DNS and ARP are exactly that.

DNS (Domain Name System) is an Application Layer Protocol used for translating URLs to IPs.
 It's used by every application layer service that needs to communicate with some server whom it
 doesn't know its address.

ARP (Address Resolution Protocol) is a Link Layer Protocol used for translating IPs to MAC addresses.
 It's used when two computers communicate on the same local network.

When two computers need to send data to each other, the OSI Model dictates that the data must go
 from the Network Layer to the Link Layer.

Since IP's are just a logical layer between routers and endpoint, when a computer reaches a
 computer at the local network level, he must go through the next layer.

DNS as a server is made up from 3 types of servers.

1. Root Level Servers - holds IPs of TLD servers
2. TLD Servers (Top Level Domain) holds IPs of other DNS servers that hold IPs of the suffix
 we've asked for (.com/.net etc)
3. Authoritative Servers - holds IPs of several servers

ARP on the other hand is just one server, set at every local network.

DNS is designed to be scalable while ARP is designed to be minimal and local.